

Mónadas en TypeScript

🏠 ■ **Elaborada por:** Dr. Roberto (class-writer) **Materia:** Paradigmas y Lenguajes de Programación 2026 — UNTDF / IDEI — IF020 **Año:** 4°
Licenciatura en Sistemas **Semana:** 4 — Clase 1 **Duración:** 120 minutos

Resumen Ejecutivo

Esta clase construye progresivamente el concepto de mónada desde la motivación práctica (encadenamiento de operaciones con efectos) hasta la implementación concreta en TypeScript, con comparación lado a lado en Clojure para cada mónada. Se implementan desde cero `Maybe<T>`, `Either<E, T>` e `IO<T>` con sus tres leyes verificadas en código. Se cierra con reconocimiento de patrones monádicos en APIs existentes (`Promise`, `Array.flatMap`, `some->`) y conexión con pipelines de IA.

Mensaje central de clase:

Una mónada es un patrón con `of` + `flatMap` + tres leyes. Ya lo usaban con `Promise.then` y `some->`. Hoy construyen la abstracción desde cero y entienden por qué funciona.

Estructura en un vistazo:

| Bloque | Tiempo | Qué logra | |---|---|---| | 1 — Motivación | 20 min | Muestra el problema del encadenamiento y define `flatMap` | | 2 — Maybe | 25 min | Implementa Maybe en TS y Clojure con pipeline completo | | 3 — Either | 25 min | Implementa Either en TS y Clojure con validador | | 4 — IO + Leyes + Ocultas | 25 min | Completa IO, verifica leyes, revela mónadas escondidas | | 5 — Ecosistemas + IA | 15 min | fp-ts/Effect, cats, pipeline IA, reflexión final | | Cierre | 10 min | Síntesis, preguntas, adelanto T06 |

Artefactos de clase:

- `filminas.md` — F-01 a F-42
- `minuta.md` — guion completo por filmina

- `guia-estudio.md` — documento de estudio autónomo para alumnos
- `guiaprofesor.md` — este documento

Índice de Artefactos

| Archivo | Descripción | Uso en clase | |---|---|---| | `diseno.md` | Alcance pedagógico y restricciones | Control de alcance (evitar desvíos) | | `minuta.md` | Guion por filmina con tiempos y transiciones | Documento operativo principal durante dictado | | `filminas.md` | Plan de presentación con código y directivas | Soporte visual de clase | | `guia-estudio.md` | Desarrollo expandido con implementaciones | Preclase, repaso y recuperación | | `guiaprofesor.md` | Síntesis ejecutiva docente | Repaso rápido previo al dictado |

Prerrequisitos del Tema

Verificar al inicio de clase que los alumnos manejan:

- `Result<T, E>` con tagged unions (Tema 04) — es el precursor de `Either`.
- `map`, `filter`, `reduce` como pipeline (Tema 03-04).
- Lectura de Clojure básica: `defn`, `let`, `->`, `some->` (Tema 04).
- Tipos genéricos TypeScript: `<T>`, `<E, T>`.

Si el dominio de `Result<T, E>` no es sólido, dedicar 5 min extra al inicio con un repaso rápido (usar F-02 como base).

Plan de Clase Detallado

BLOQUE 1 — Motivación: ¿por qué mónadas? (20 min)

Filminas: F-01 a F-07 (incluyendo F-06b, F-06c)

Objetivo del bloque: Crear la necesidad de `flatMap` antes de nombrar "mónada". La palabra "mónada" debe aparecer recién ~minuto 15.

Estrategia pedagógica: Partir del código feo (if/null anidados) que los alumnos ya escriben. Mostrar el patrón repetido en TS y Clojure. Dar la analogía intuitiva (el sobre certificado) ANTES de cualquier definición técnica. Luego construir la abstracción inductivamente.

- **F-02 a F-04:** Código roto en ambos lenguajes. No dar la solución todavía — dejar que sientan la incomodidad.
- **F-05:** Tabla que cristaliza el patrón común. Preguntar: "¿Qué tienen en común estas dos versiones?"
- **F-06 ■ NUEVA — CLAVE:** Explicación intuitiva con analogía del sobre certificado. Esta filmina baja el concepto a tierra ANTES de la formalización. Dedicarle tiempo: 5 min mínimo.
- **F-06b:** Código lado a lado: sin mónadas vs con mónadas. Impacto visual.

- **F-06c:** Parte técnica: por qué `map` no alcanza y `flatMap` sí. Diagrama del doble envoltorio.
- **F-07:** Analogía del contenedor + tabla de las tres cajas (Maybe, Either, IO) como adelanto.

Puntos de cuidado:

- NO arrancar con "una mónada es un endofunctor en la categoría..." — eso mata la motivación.
- NO decir "es simple" — para muchos no lo es. Decir "es un patrón que se repite".
- SÍ dedicar tiempo generoso a F-06 (analogía) — es la filmina que desbloquea la comprensión.
- SÍ hacer pausas para preguntas después de F-06c — es el concepto más denso del bloque.

Filminas: F-08 a F-14

Objetivo del bloque: Que los alumnos vean Maybe construida desde cero en TS, la reconozcan en Clojure, y comparen ambos enfoques.

Flujo sugerido:

1. **F-08 a F-10 (10 min):** TypeScript — definición → operaciones → pipeline completo. Ejecutar en REPL/consola en vivo. Mostrar ambos caminos (Just y Nothing).
2. **F-11 a F-12 (10 min):** Clojure — `some->` primero (idiomático), luego `cats/maybe` (formal). Ejecutar en REPL.
3. **F-13 a F-14 (5 min):** Comparativa. `some->` vs `cats` y tabla TS vs Clojure.

Actividad del bloque (codificación guiada): Usar el ejemplo de `findUser` → `getAddress` → `getPostalCode`. Los alumnos deben intentar escribir el `flatMap` antes de ver la solución. Dar 2-3 min para que intenten y luego mostrar.

Preguntas frecuentes anticipadas:

- "¿No es más fácil usar `?.?`?" → Sí para acceso a propiedades, no para funciones que pueden fallar. `?.` es Maybe ultra-simplificado.
- "¿En Clojure no se usa `cats` en producción?" → Rara vez. Los equipos prefieren `some->` y convenciones. Pero conceptualmente es lo mismo.

BLOQUE 3 — Either (25 min)

Filminas: F-15 a F-22

Objetivo del bloque: Que vean Either como "Maybe con razón del fallo", la implementen en TS, y comparen con Clojure (cats y mapas idiomáticos).

Flujo sugerido:

1. **F-15 a F-17 (10 min):** TypeScript — tipo, operaciones, validador de formulario. Ejecutar en vivo.
2. **F-18 (3 min):** Either vs try/catch — establecer el criterio pragmático.
3. **F-19 a F-20 (8 min):** Clojure — `cats/either` y luego mapas `{:ok/:error}`. Contrastar la ergonomía.
4. **F-21 a F-22 (4 min):** Discusión datos vs tipos + tabla comparativa.

Actividad del bloque (codificación guiada): Validador de formulario con 4 campos. Dar las funciones de validación individuales pre-escritas. Los alumnos deben componer el pipeline con `flatMap`. Luego ver la versión Clojure.

Punto de cuidado didáctico:

- El salto de Maybe a Either es pequeño (agregar tipo de error), pero los alumnos pueden confundirse. Enfatizar: "Either = Maybe + información del fallo".
- La filmina F-21 (datos vs tipos) puede generar debate. Moderar hacia: "no hay respuesta correcta universal — depende del contexto".

BLOQUE 4 — IO, leyes y mónadas ocultas (25 min)

Filminas: F-23 a F-32

Objetivo del bloque: Completar las tres mónadas canónicas con IO, verificar leyes, y revelar mónadas en APIs conocidas.

Flujo sugerido:

1. **F-23 a F-24 (6 min):** IO en TypeScript — tipo, pipeline. Enfatizar: nada se ejecuta sin `.run()`.
2. **F-25 a F-26 (5 min):** IO en Clojure — no necesaria. `delay`, `thunks`, `core.async`.
Tabla.
3. **F-27 a F-29 (9 min):** Leyes monádicas — enunciado, verificación en TS y Clojure. Este es el subbloque más abstracto — ir lento.
4. **F-30 a F-31 (3 min):** Promise como mónada, más ejemplos ocultos. Momento de "revelación".

5. **F-32 (2 min):** Jerarquía Functor → Monad. Solo visión general, no profundizar.

Punto de cuidado didáctico:

- Las leyes son el punto más abstracto de la clase. Usar la analogía: "son como las leyes de la aritmética — no las verificás cada vez que sumás, pero sin ellas la suma no funcionaría".
- El momento de revelar que `Promise.then` es un `flatMap` suele generar un "¡ahh!" colectivo. Aprovecharlo para fijar el concepto.

BLOQUE 5 — Ecosistemas, IA y reflexión (15 min)

Filminas: F-33 a F-38

Objetivo del bloque: Conectar lo construido a mano con el ecosistema industrial y cerrar con reflexión comparativa.

Flujo sugerido:

1. **F-33 a F-34 (5 min):** fp-ts, Effect, cats, manifold. Mostrar código, no profundizar. Mensaje: "esto ya existe industrializado".

2. **F-35 (3 min)**: Pipeline IA con mónadas. Conectar: "es el mismo flatMap del validador".
3. **F-36 (4 min)**: Discusión abierta — ¿mónadas en Clojure?. Moderar las tres preguntas.
4. **F-37 a F-38 (3 min)**: Tablas de síntesis y guía de decisión.

Para la discusión (F-36): Preparar argumentos de las dos orillas:

- A favor de mónadas formales: disciplina, comunicación de intenciones, refactoring seguro.
- A favor de datos simples: pragmatismo, menos ceremonia, filosofía Clojure.
- Guiar hacia: "el concepto es universal; la forma de expresarlo depende del lenguaje".

CIERRE (10 min)

- **F-39 (4 min):** Recapitulación — repasar los 7 puntos del diagrama.
- **F-40 (3 min):** 5 preguntas clave. Esperar respuestas antes de dar las propias.
- **F-41 (2 min):** Adelanto T06 — FP en Python.
- **F-42 (1 min):** Cierre con la frase: "ya las conocían, ahora saben por qué funcionan".

Extractos Clave de la Bibliografía

Wadler (1995) — *Monads for Functional Programming*

"Shall I be pure or impure? (...) Pure languages are defined by what they lack — the ability to change state — and so they might seem impoverished."

Wadler muestra que las mónadas resuelven tres problemas en lenguajes puros: excepciones, estado, y output. La construcción es inductiva: primero un evaluador simple, luego se agrega cada efecto con mónadas sin cambiar la estructura base.

Uso en clase: Cuando explica IO, mencionar que Wadler demostró en 1995 que este patrón unifica todos los efectos. No es una invención ad-hoc.

Anderlind & Åsberg (2023) — *Monadic Programming in Imperative Languages*

Tesis de Chalmers que implementa Maybe, Either y IO en Rust y C#, y evalúa si el patrón mejora la calidad del código en lenguajes imperativos. Conclusión: las

mónadas mejoran composición y manejo de errores, pero requieren sobrecarga conceptual que no todos los equipos aceptan.

Uso en clase: Cuando algún alumno pregunte "¿esto se usa en la industria?", citar esta tesis: "se evaluó en lenguajes imperativos y se encontró que mejora composición pero requiere formación del equipo".

Señales de Alerta

| Señal | Acción | |---|---| | Alumnos confundidos en F-06c (map vs flatMap) | Parar. Hacer ejemplo en la pizarra con cajas: map anida, flatMap aplana. Repetir con otro ejemplo. | | "¿Y esto para qué sirve?" | Volver a F-02: el código con if/null anidados. "¿Prefieren esto o el pipeline con flatMap?" | | Discusión TS vs Clojure se alarga | Cortar con: "no hay lenguaje mejor — hay compromisos distintos. Avanzamos y vuelven al tema en la autoevaluación." | | Alumnos piensan que todo debe ser monádico | F-38: "Usá mónadas cuando el error o efecto es parte del dominio. No para sumar dos números." | | Las leyes monádicas generan desconexión | Usar analogía aritmética: "sin asociatividad, mover paréntesis en una suma cambia el resultado. Lo mismo con flatMap." |

Preguntas Esperadas y Respuestas Sugeridas

"¿Maybe es lo mismo que Optional de Java?" → Sí, conceptualmente es lo mismo. Java lo llama `Optional<T>`, Rust lo llama `Option<T>`, Haskell `Maybe a`. Diferentes nombres, mismo patrón.

"¿Por qué no usar try/catch para todo?" → `try/catch` es para errores *excepcionales* (disco lleno, OOM). `Either` es para errores *del dominio* (validación, not found). Con `try/catch` el tipo del error es `unknown` — con `Either` es `E` verificado.

"¿Promise.then viola las leyes?" → En la práctica no. Técnicamente, la evaluación `eager` de `Promise` hace que no sea una mónada "pura" (una `IO lazy` sí lo es). Pero para efectos prácticos y a nivel de tipos, se comporta como mónada.

"¿En Clojure no se puede hacer tipado estático?" → Existe `clojure.spec` y `malli` para validación de formas de datos. No es tipado estático como `TypeScript`, pero permite contratos verificables en runtime.

"¿Qué son monad transformers?" → Permiten combinar mónadas (ej: `MaybeT( IO(a) )` para un `IO` que puede no tener valor). Están fuera del alcance de este tema — mencionarlos como lectura complementaria.

Conexiones con Otros Temas

| Tema | Conexión | |---|---| | T03 — Intro Funcional | `map/filter/reduce` son la base; `flatMap` es el siguiente paso | | T04 — FP Avanzado | `Result<T, E>` es `Either` con

otro nombre; composición de funciones | | T06 — FP en Python | `Optional`, pattern matching (3.10+), monads en Python (returns lib) | | TP-05 | Implementar Maybe + Either con tests de leyes |

Checklist Pre-Clase

- ☐ REPL TypeScript configurado (ts-node o tsx)
- ☐ REPL Clojure configurado (con `cats` en deps)
- ☐ Código de las 3 mónadas pre-cargado (para copiar si se complica el live coding)
- ☐ Proyector/pantalla preparada para mostrar código y filminas
- ☐ Verificar que `cats` funciona en el REPL: `(require '[cats.monad.maybe :as m])` sin error
- ☐ Leer minuta.md completa como ensayo mental de la clase